

**MBA
USP
ESALQ**

**ÁRVORES, REDES E ENSEMBLE
MODELS IV**

João F. Serrajordia R. de Mello

***A responsabilidade pela idoneidade, originalidade e licitude dos conteúdos didáticos apresentados é do professor.**

Proibida a reprodução, total ou parcial, sem autorização. Lei nº 9610/98

Você vai precisar de...



Preparativos

- Abrir o R
- Importar as bibliotecas
- Algo para fazer suas anotações

Agenda

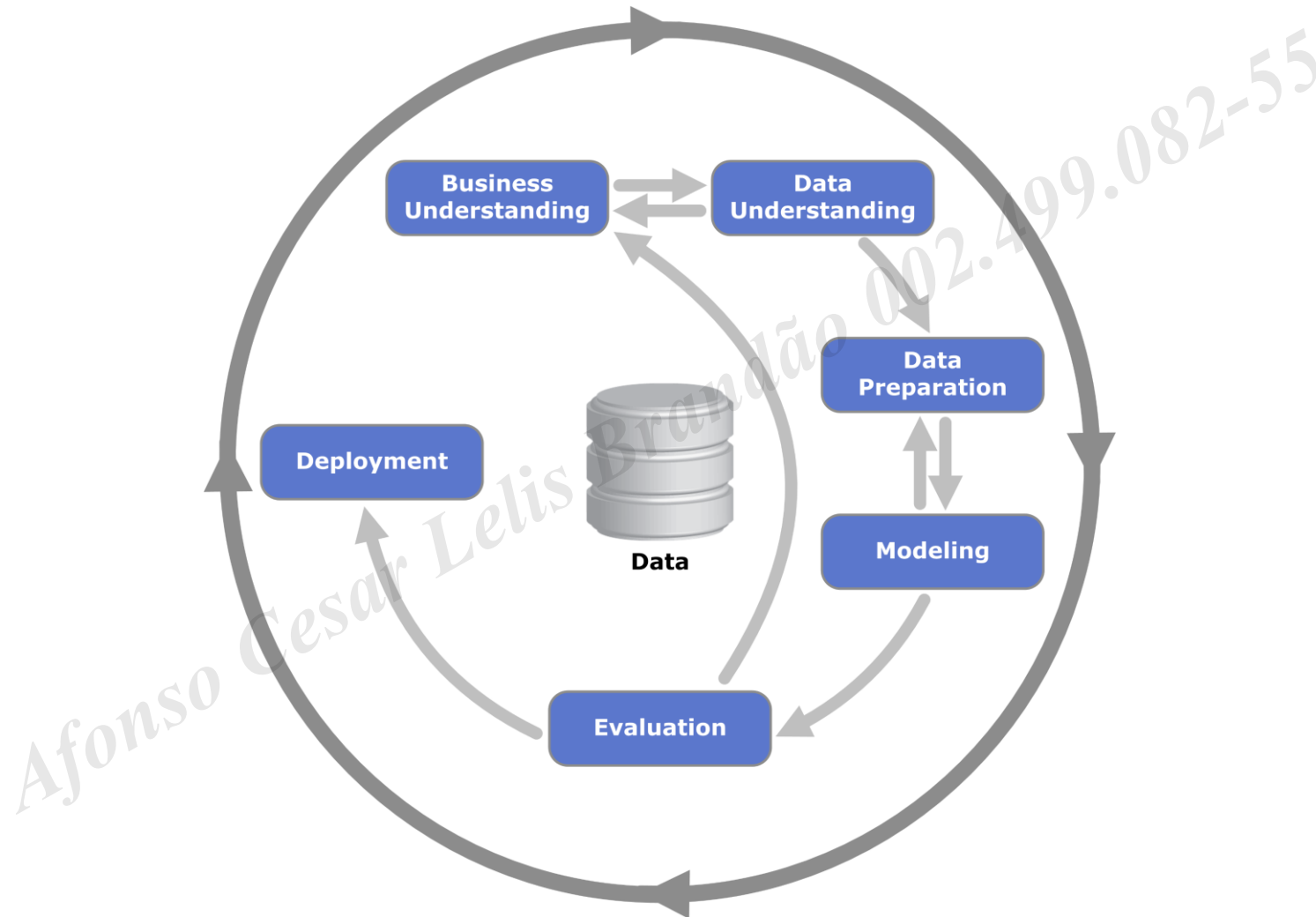
Revisão

Histórico

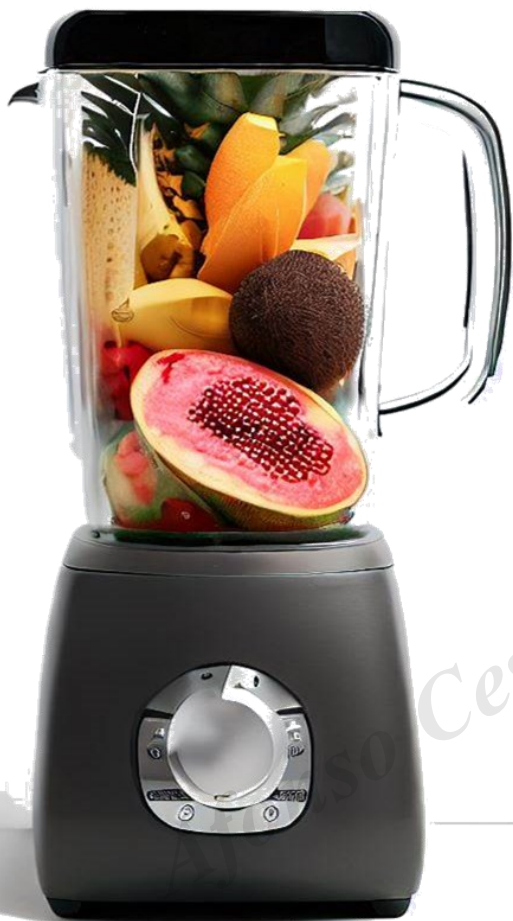
Ideias básicas

Usos

CRISP-DM



Fonte: <https://www.the-modeling-agency.com/crisp-dm.pdf>



Ensemble

Um ensemble é qualquer mistura de modelos já existentes.
Os principais tipos são:

Bagging

Boosting

Stacking

Classificação

Classificação dos algoritmos



Supervisionados

- Regressão
- GLM
- GLMM
- Support vector machines
- Naive Bayes
- K-nearest neighbors
- Redes Neurais
- Decision Trees



Não supervisionados

- K-Means
- Métodos hierárquicos
- Mistura Gaussiana
- DBScan
- Mini-Batch-K-Means

Estamos aqui!

Classificação dos algoritmos



Resposta contínua

- Regressão
- GLM
- GLMM
- Support vector machines
- K-nearest neighbors
- Redes Neurais
- Regression Trees



Resposta discreta

- Regressão logística
- Classification trees
- Redes Neurais
- GLM
- GLMM

Estamos aqui!

Classificação dos algoritmos

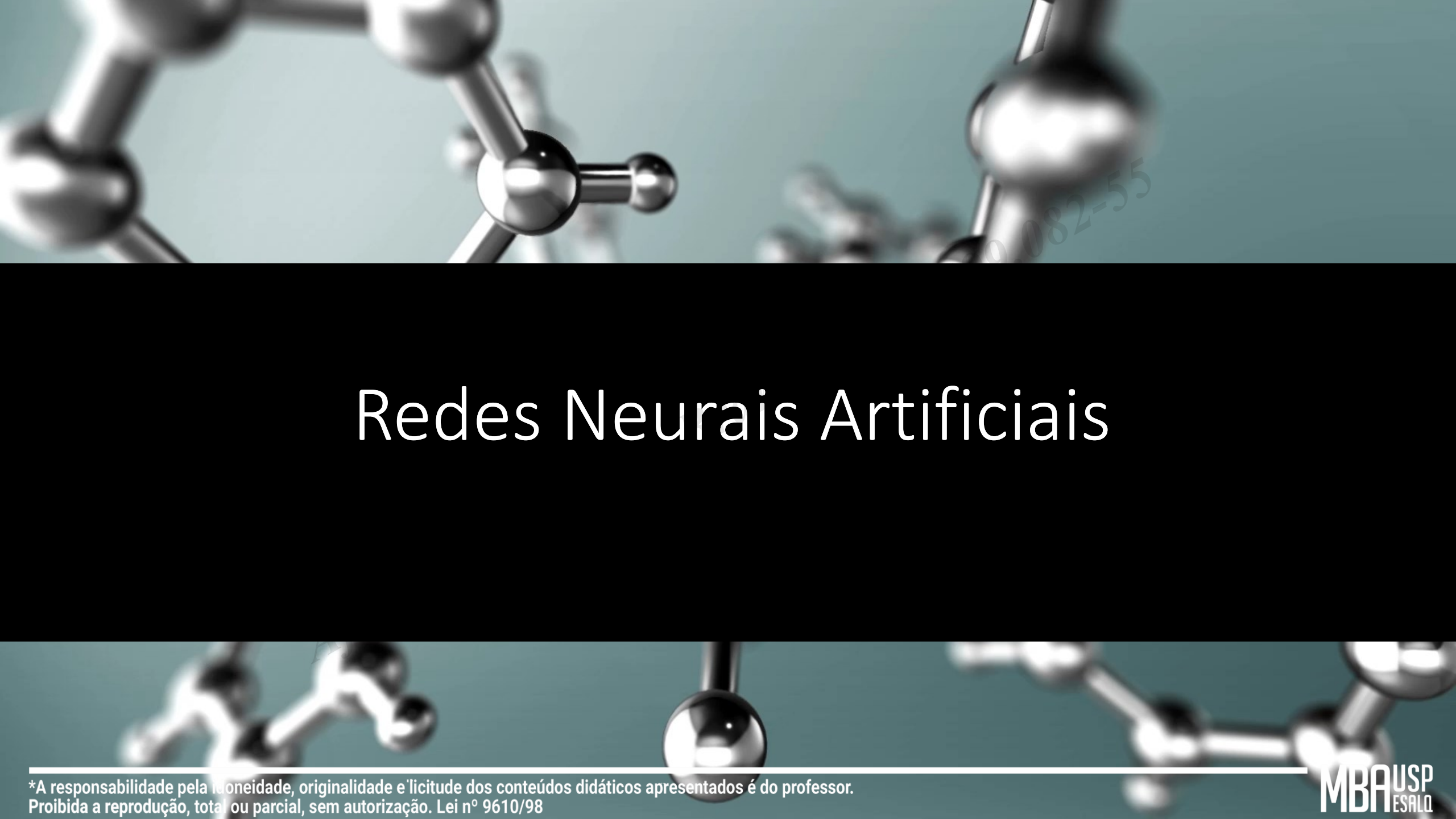
Métodos Machinelânicos

- Árvores de decisão
- Bagging
- Boosting
- K-NN
- Redes Neurais
- Support Vector Machines

Métodos Machinelânico- estatísticos

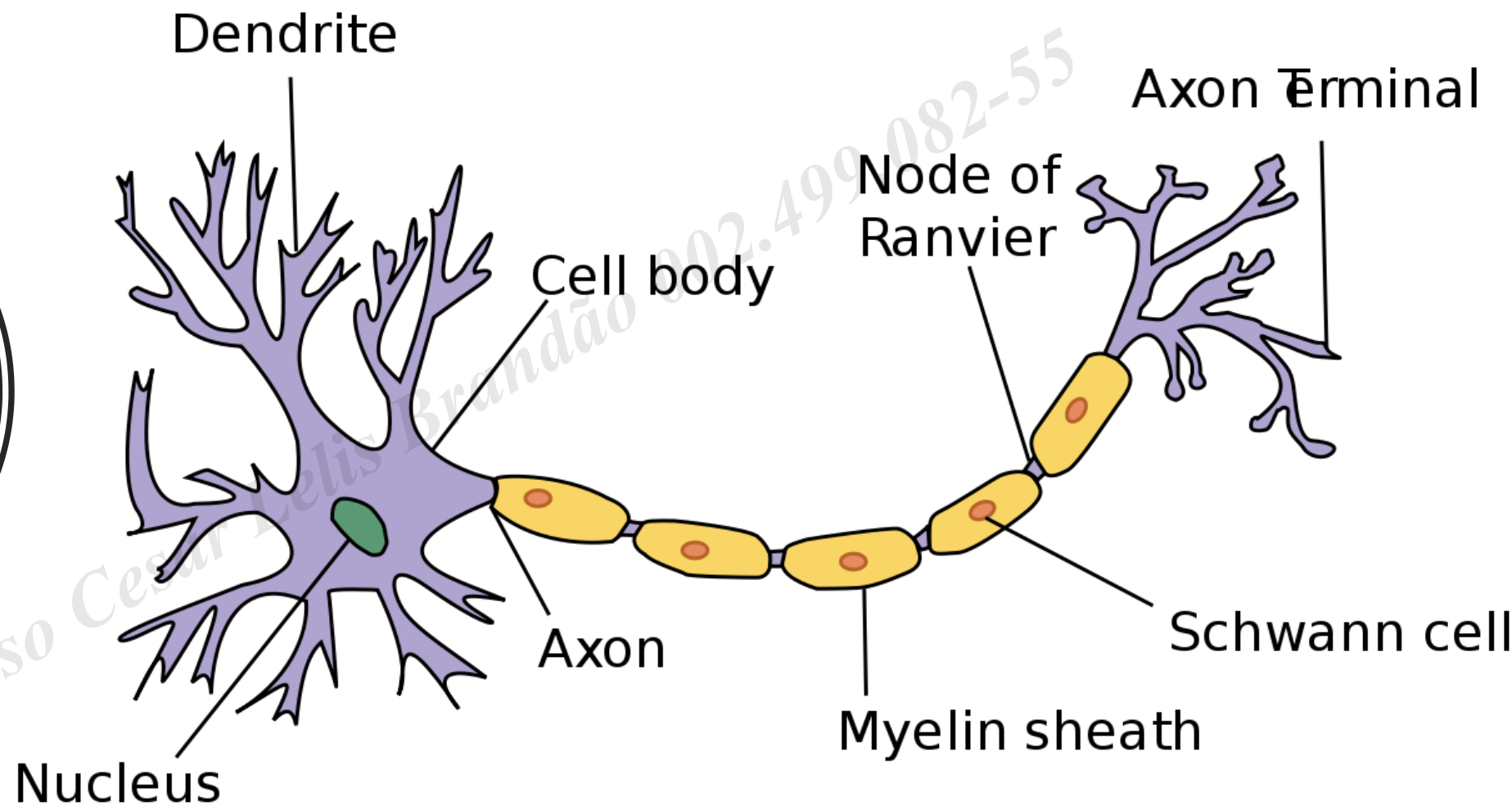
- Regressão
- GLM
- GLMM
- ANOVA

Estamos aqui!



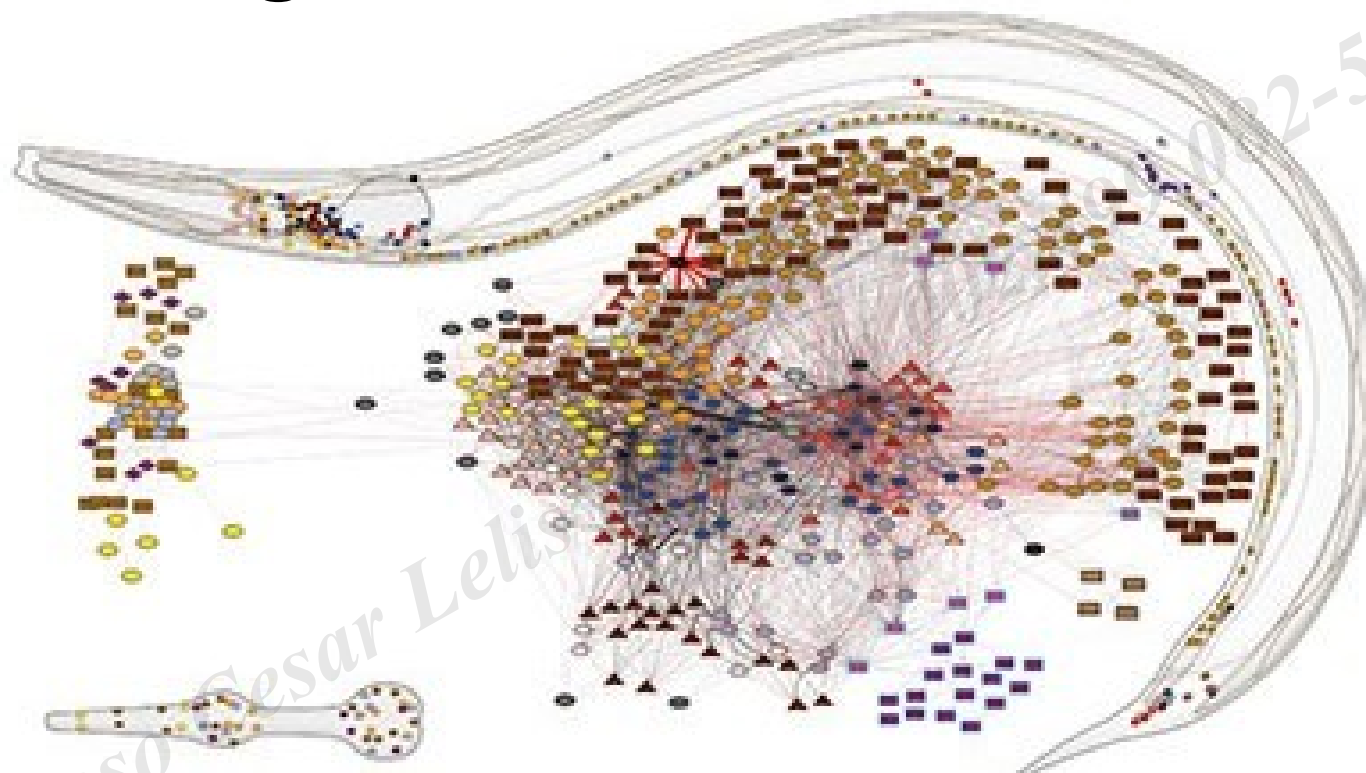
Redes Neurais Artificiais

Metáfora



<https://en.wikipedia.org/wiki/Myelin>

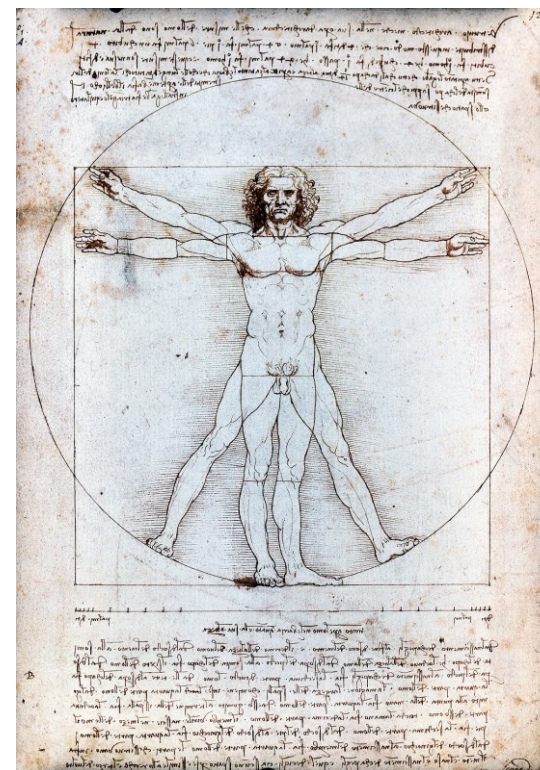
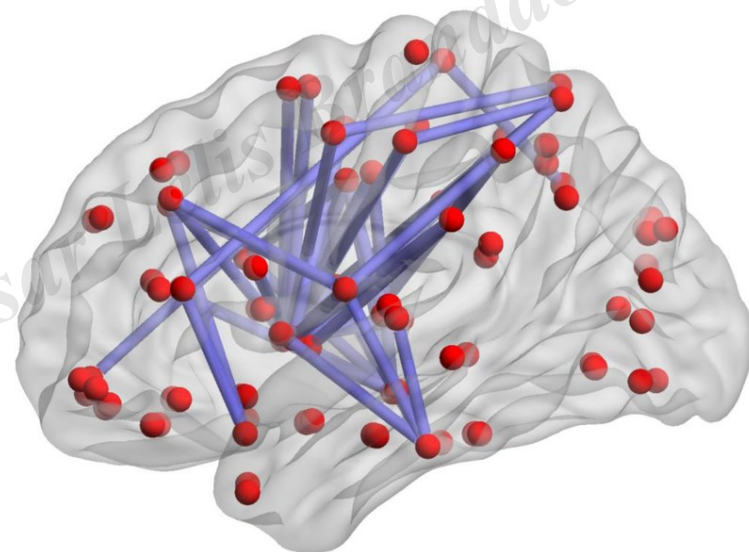
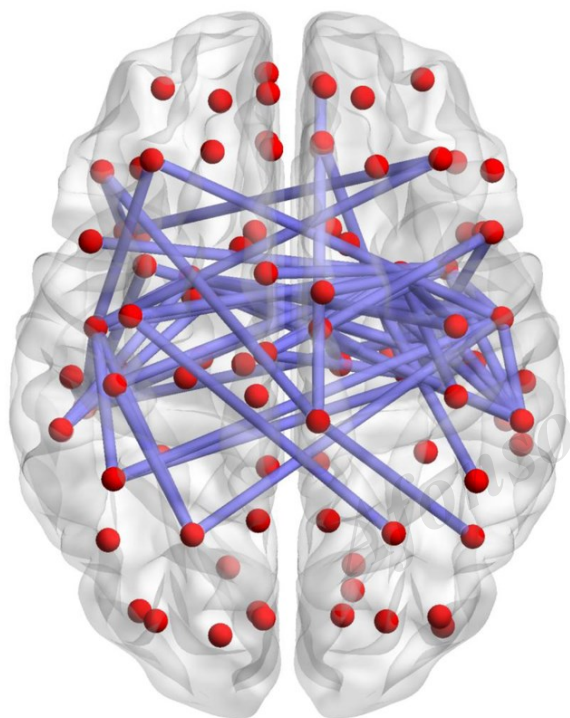
Exemplo biológico



- *Nematelminto*: 302 neurônios

Rede Neural Humana

- *Homo sapiens*: 100.000.000.000 de neurônios



Onde vivem?



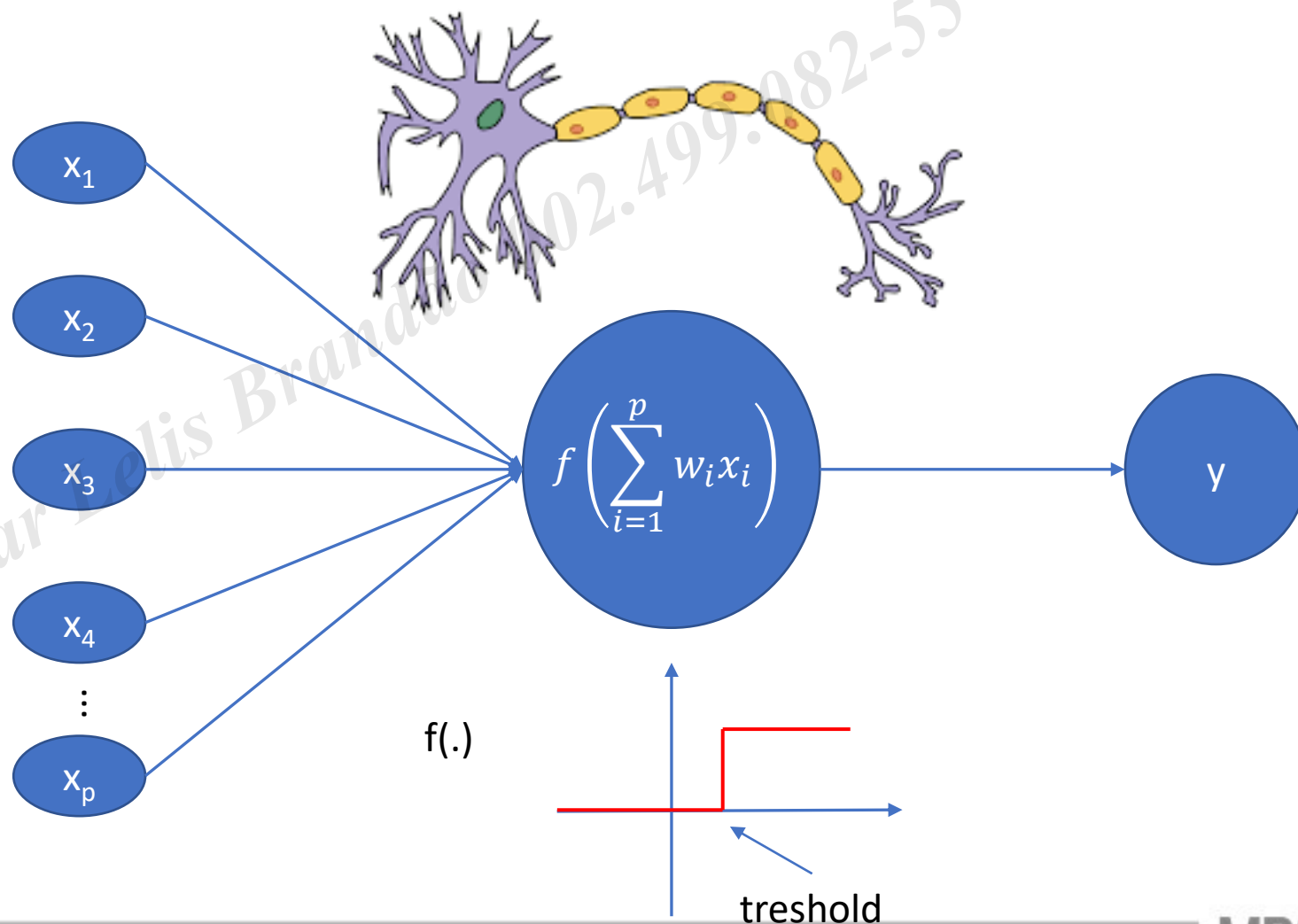
Redes Neurais Artificiais têm tido muito sucesso em problemas com dados pouco estruturados como imagens, áudios, textos e vídeos.

Neurônio de McCulloch-Pitts

É um modelo da década de 60/70, mas com muita influência nos mais modernos.

O perceptron consolida informações de estímulos iniciais, das variáveis $x_1, x_2, \dots, x_p$ através de uma combinação linear (soma ponderada).

Devolve uma 'decisão' binária, de sim ou não, caso a combinação dos estímulos supere um certo valor limite (*threshold*).



Exemplo de funcionamento

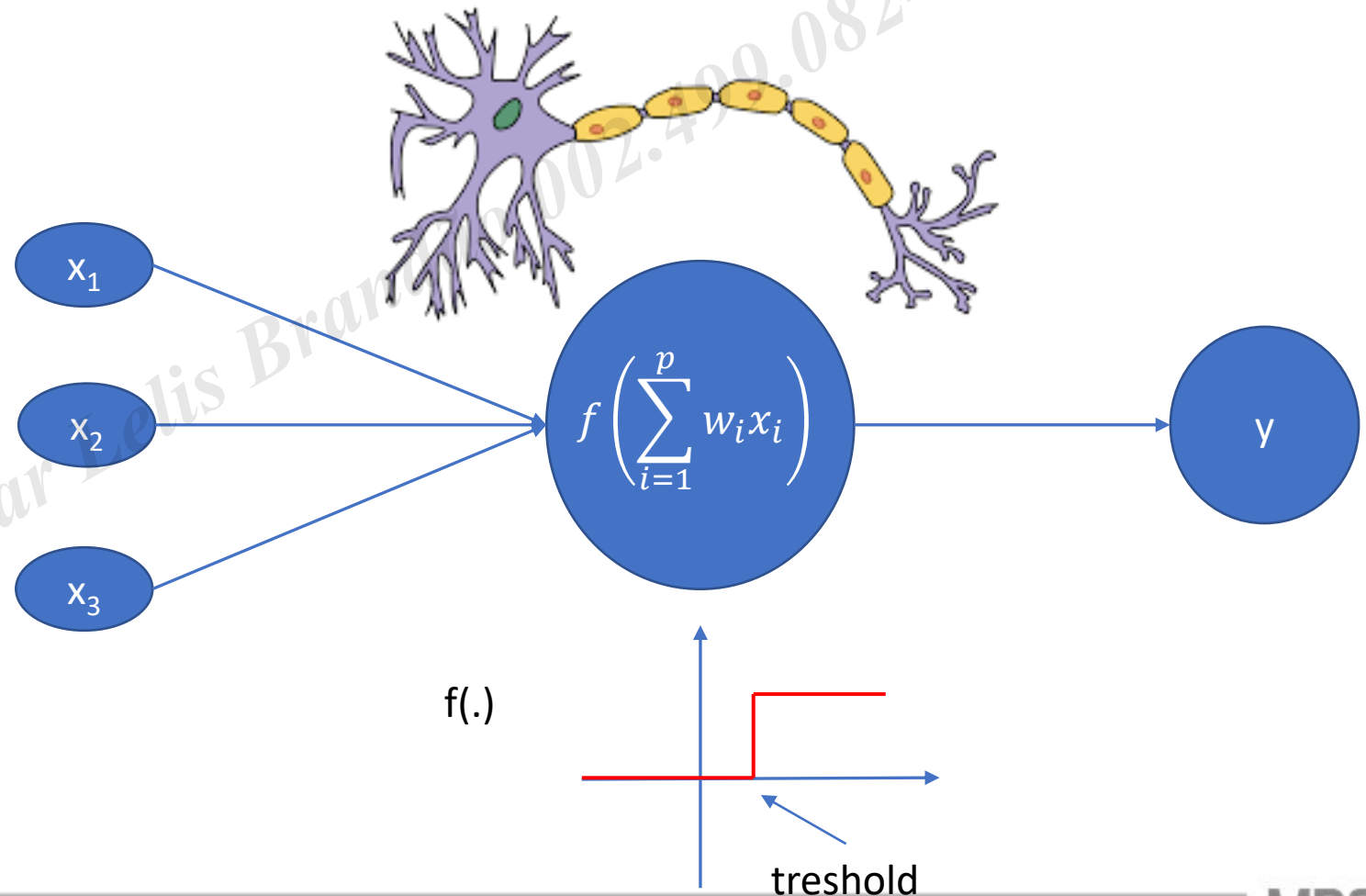
Vou no jogo do Bragantino?

Considere três fatores para esta decisão, cada fator é binário, 0=não, 1=sim:

- 1) Se está chovendo
- 2) Se vai ser televisionado
- 3) Se é final do campeonato

Cada fator tem uma ponderação, digamos: 3, -1 e 6 respectivamente.

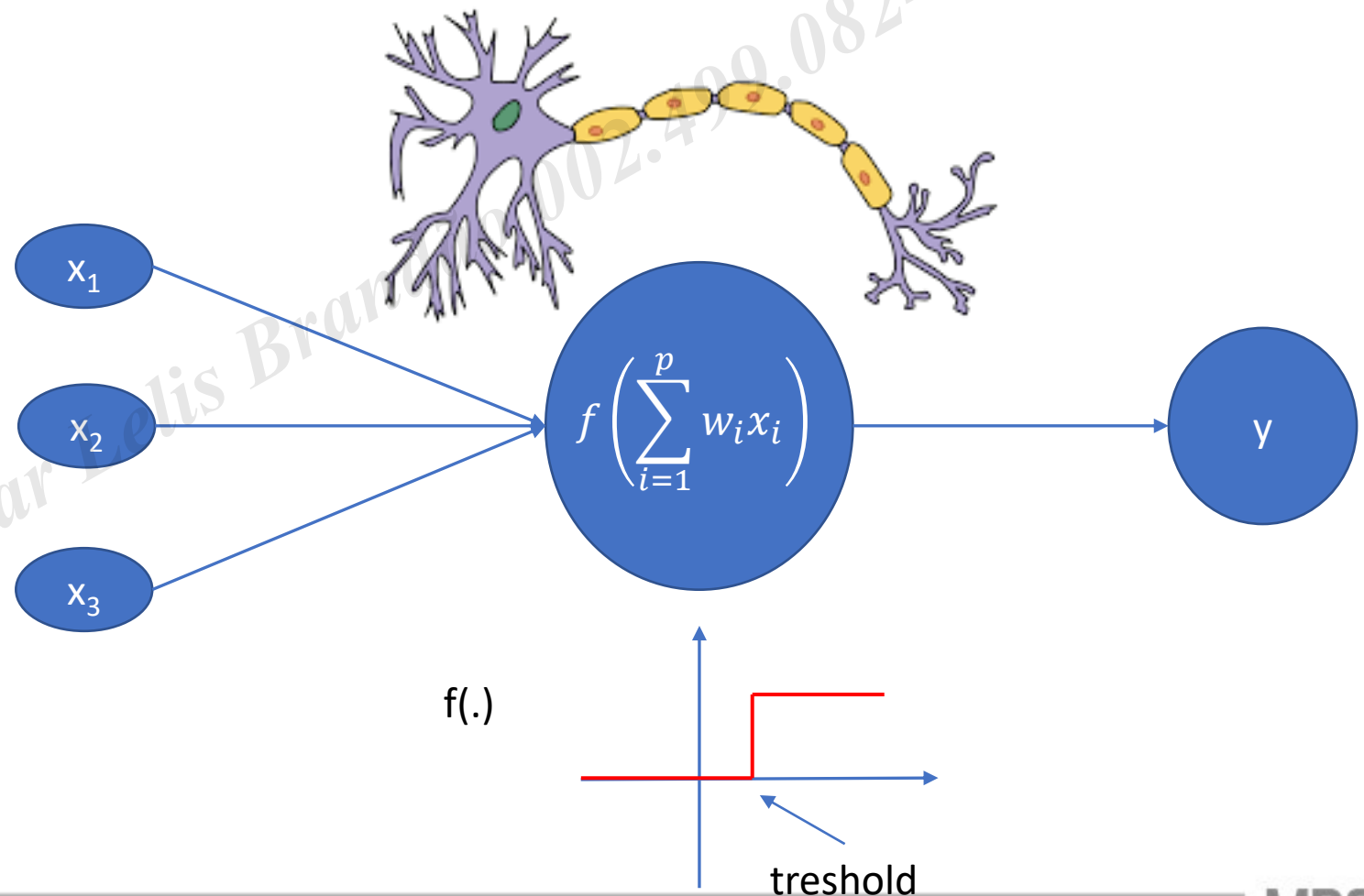
Se a soma desses fatores passar de um valor limite (digamos 3), então a decisão é ir no jogo.



Perceptron binário?

Mas a condição do tempo é mesmo binária? Ou é uma coisa mais contínua, passando de um céu azul limpinho, por um dia nublado ou um chuvisquinho a uma tempestade com direito a relâmpagos?

A decisão igualmente pode ser convicta para o sim ou para o não, ou pode estar mais numa zona de indecisão.



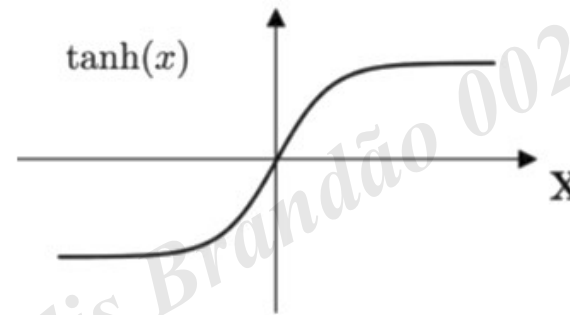
Funções de ativação

Cada neurônio da RNA é uma função não linear de uma combinação linear dos elementos da camada anterior.

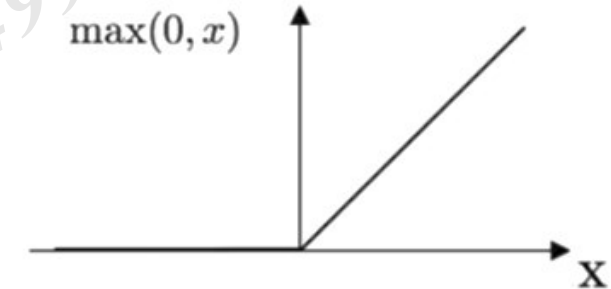
Essa função é chamada de função de ativação.

Na analogia com a rede neural natural, ele visa identificar a presença ou não de um estímulo específico.

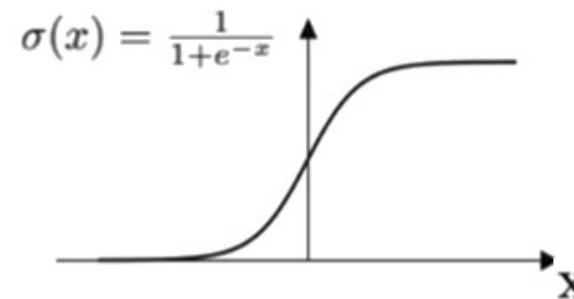
Tanh



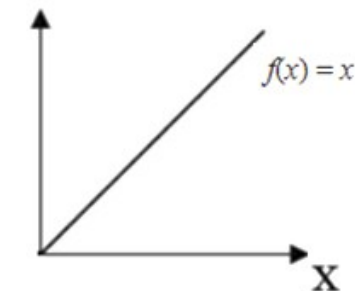
ReLU



Sigmoid



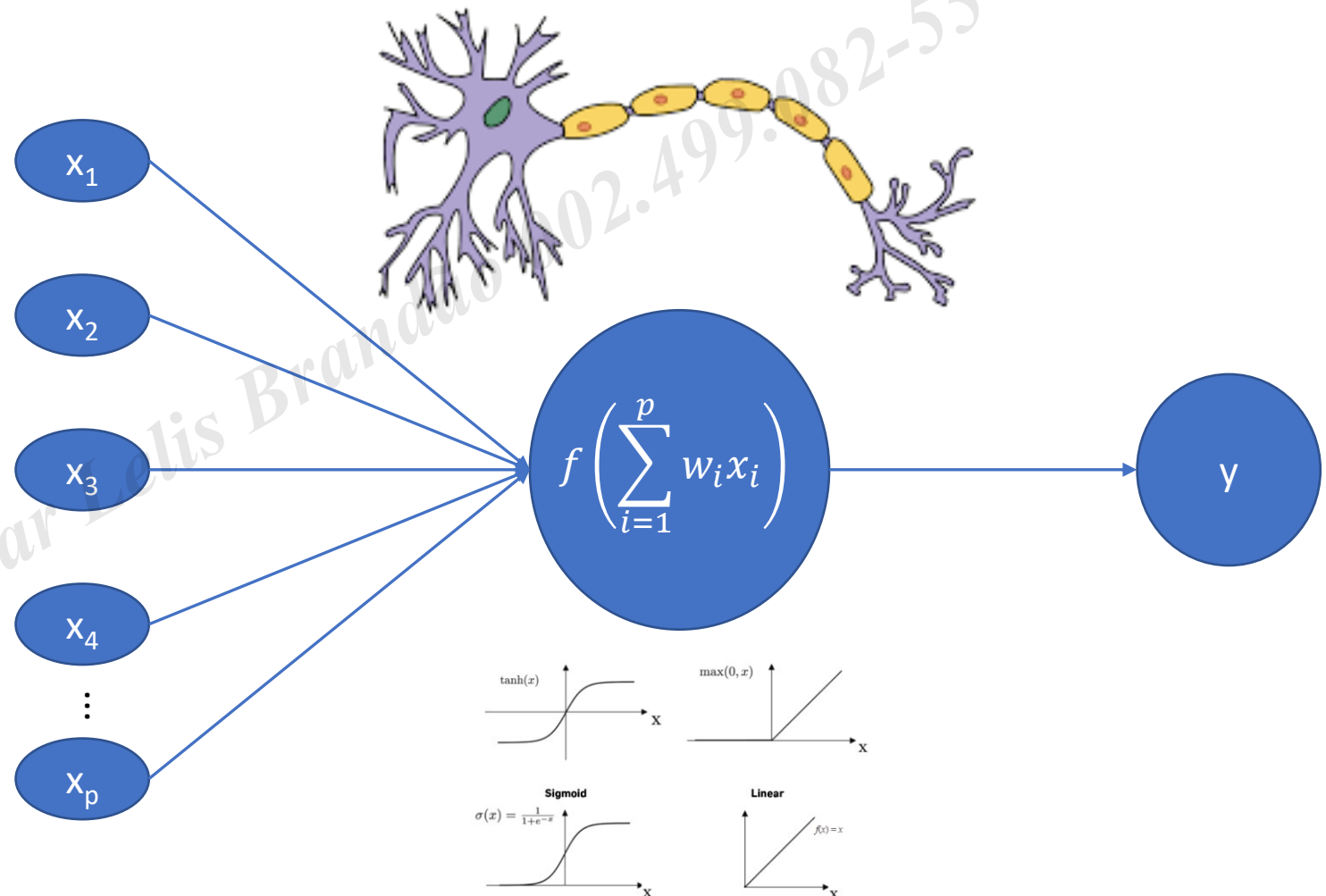
Linear



Perceptron

O perceptron é um tipo de neurônio bastante comum nas RNAs.

Eles podem ainda ser combinados de formas diferentes formando diferentes arquiteturas de RNAs.



Combinações de perceptrons

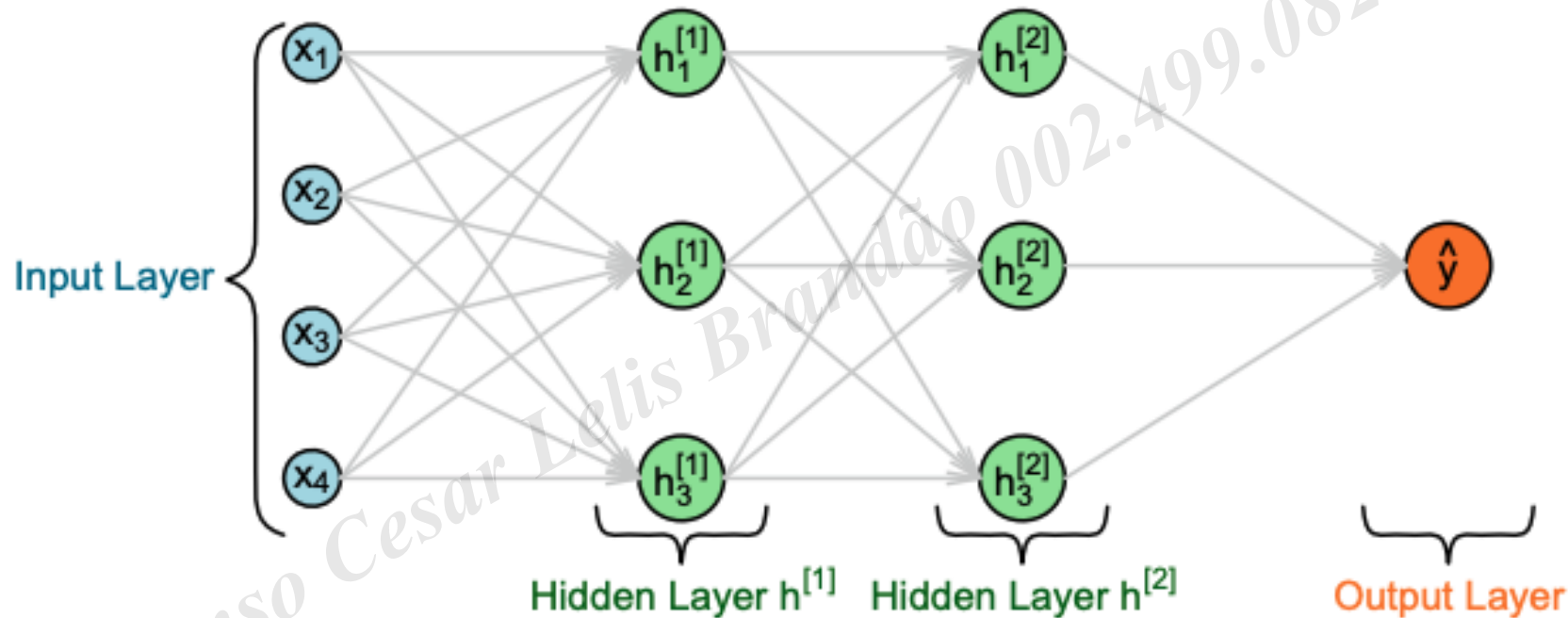


Fig. 2.3 A representation of a neural network with four input features, two hidden layers with three nodes each, and an output layer

Deep learning with R - Abhijit Ghatak, ed. Springer, 2019

Como roda no R?

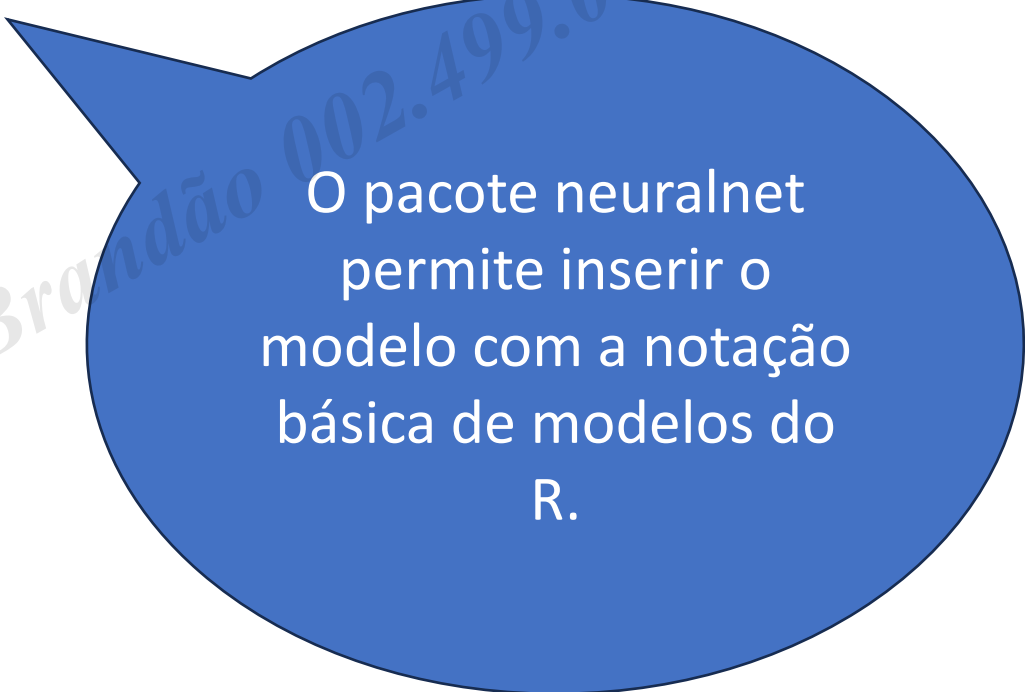
- Há várias implementações de RNA disponíveis.
- Vamos utilizar o pacote 'neuralnet' do R

Como roda no R?

```
rn0 <- neuralnet::neuralnet(y ~ x,  
  data=df1,  
  threshold = 0.01,  
  act.fct = 'logistic'  
)
```

```
plot(rn0)
```

```
df1['pred0'] <- predict(rn0, df1)
```



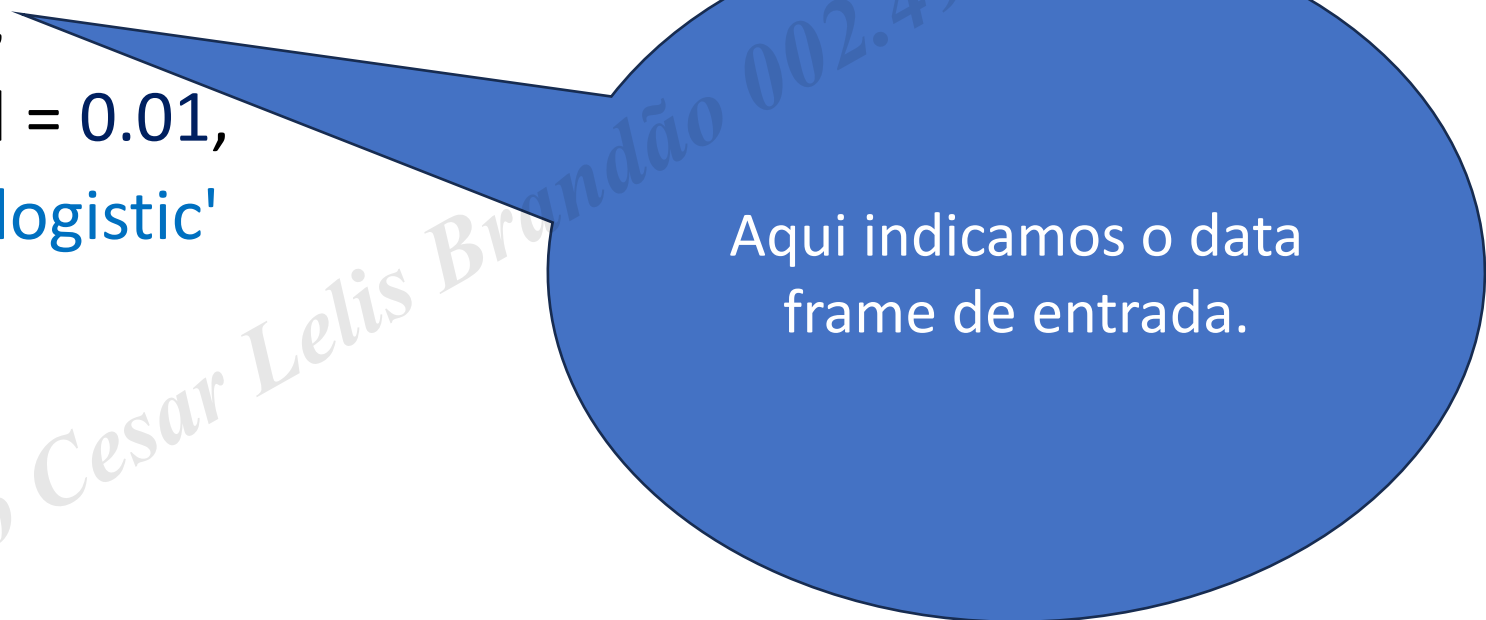
O pacote neuralnet
permite inserir o
modelo com a notação
básica de modelos do
R.

Como roda no R?

```
rn0 <- neuralnet::neuralnet(y ~ x,  
  data=df1,  
  threshold = 0.01,  
  act.fct = 'logistic'  
)
```

```
plot(rn0)
```

```
df1['pred0'] <- predict(rn0, df1)
```



Aqui indicamos o data
frame de entrada.

Como roda no R?

```
rn0 <- neuralnet::neuralnet(y ~ x,  
  data=df1,  
  threshold = 0.01,  
  act.fct = 'logistic'  
)
```

```
plot(rn0)
```

```
df1['pred0'] <- predict(rn0, df1)
```

Este é um
hiperparâmetro de
'parcimônia' da rede.

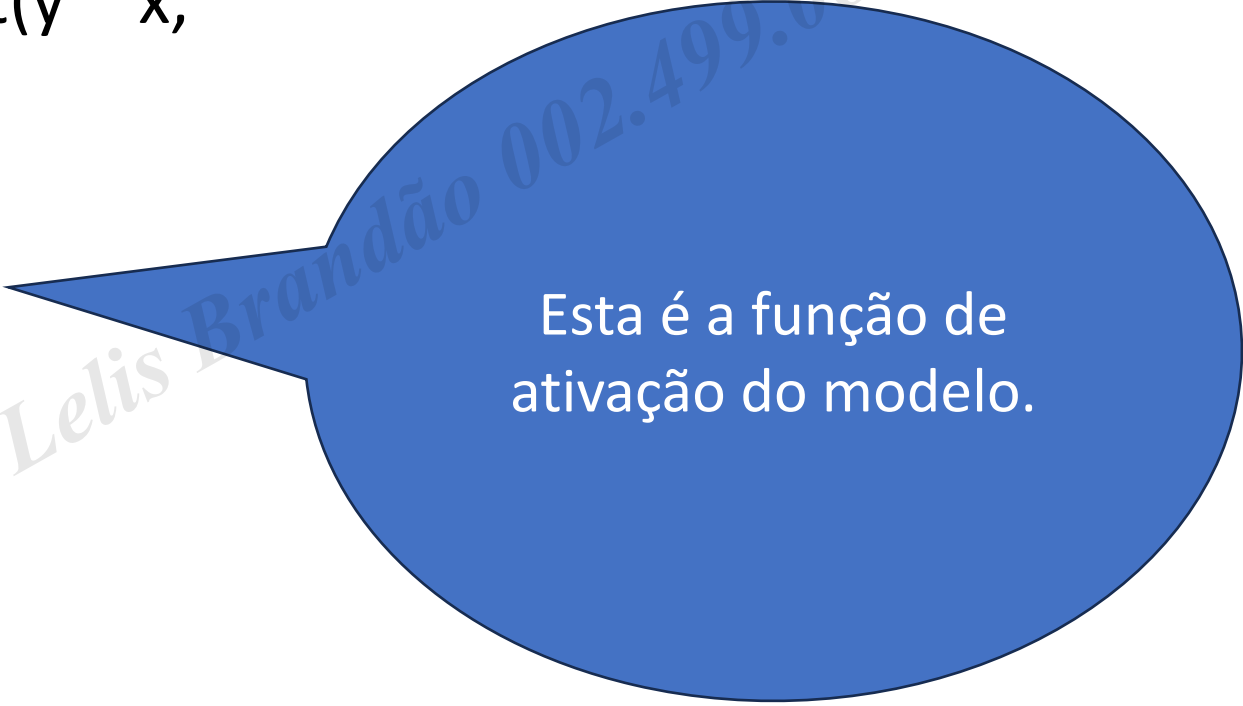
Ele tem o papel de
limitar o *overfitting* do
modelo.

Como roda no R?

```
rn0 <- neuralnet::neuralnet(y ~ x,  
  data=df1,  
  threshold = 0.01,  
  act.fct = 'logistic'  
)
```

```
plot(rn0)
```

```
df1['pred0'] <- predict(rn0, df1)
```



Esta é a função de ativação do modelo.

Como roda no R?

```
rn0 <- neuralnet::neuralnet(y ~ x,  
  data=df1,  
  threshold = 0.01,  
  act.fct = 'logistic'  
)
```

```
plot(rn0)
```

```
df1['pred0'] <- predict(rn0, df1)
```

O pacote neuralnet é compatível com a função plot. Ele mostra um grafo representando a rede, com pesos etc.

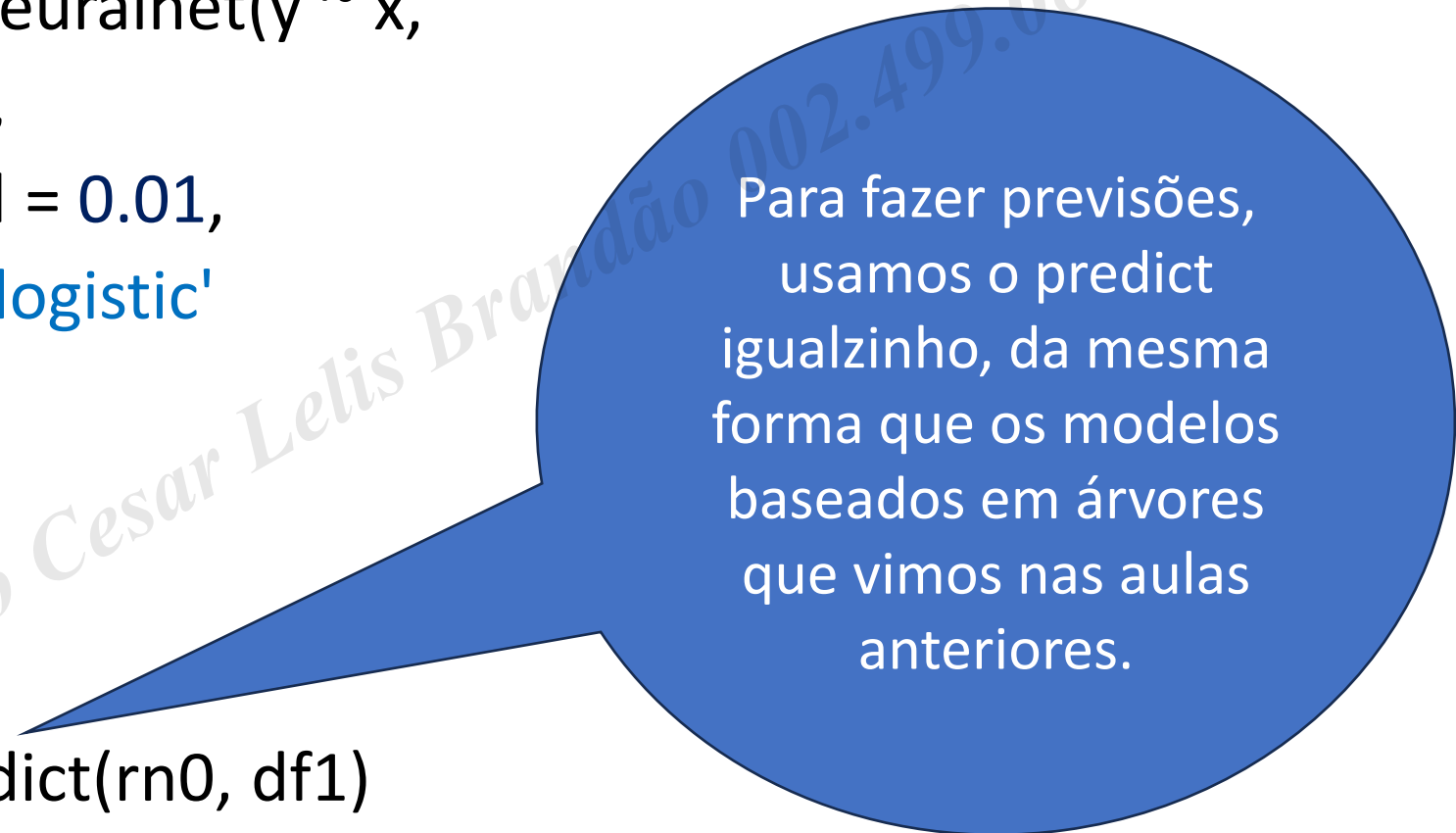
Cuidado com redes muito grandes.

Como roda no R?

```
rn0 <- neuralnet::neuralnet(y ~ x,  
  data=df1,  
  threshold = 0.01,  
  act.fct = 'logistic'  
)
```

```
plot(rn0)
```

```
df1['pred0'] <- predict(rn0, df1)
```



Para fazer previsões,
usamos o predict
igualzinho, da mesma
forma que os modelos
baseados em árvores
que vimos nas aulas
anteriores.



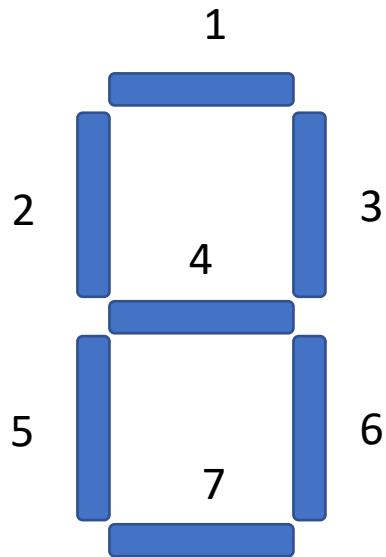
AREM4_script01_RNA_ilustracao.R

OCR – Optical Character Recognition



Vamos pensar em uma versão bem simples do problema. Dígitos de um rádio relógio antigo possuem uma estrutura bem simples.

OCR – Optical Character Recognition



Há 7 regiões básicas, que podem estar ativas ou inativas, e definem um dígito.

Por exemplo, se somente as regiões 1, 3 e 6 estão ativadas, temos o número 7.

É relativamente fácil imaginar um modelo ou uma regra, que toma essas 7 variáveis como entrada, e retorna que número está sendo mostrado. Aliás, essa tecnologia foi desenvolvida com o exato raciocínio oposto.

Perceptron de Rosenblatt

- O Perceptron de Rosenblatt (~1950-1960) tem essa ideia, só que com propósito mais geral
- foi construído para fazer OCR (optical character recognition)
- Para isso, mapeia regiões de uma imagem “ativadas” e “não ativadas”
- Cada unidade é um neurônio de McCulloch-Pitt

FIG. 1 — Organization of a biological brain. (Red areas indicate active cells, responding to the letter X.)

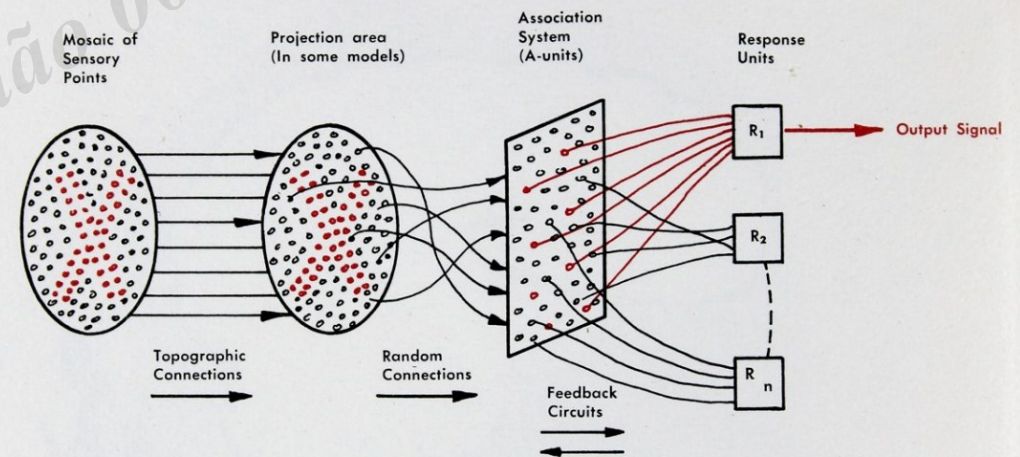


FIG. 2 — Organization of a perceptron.

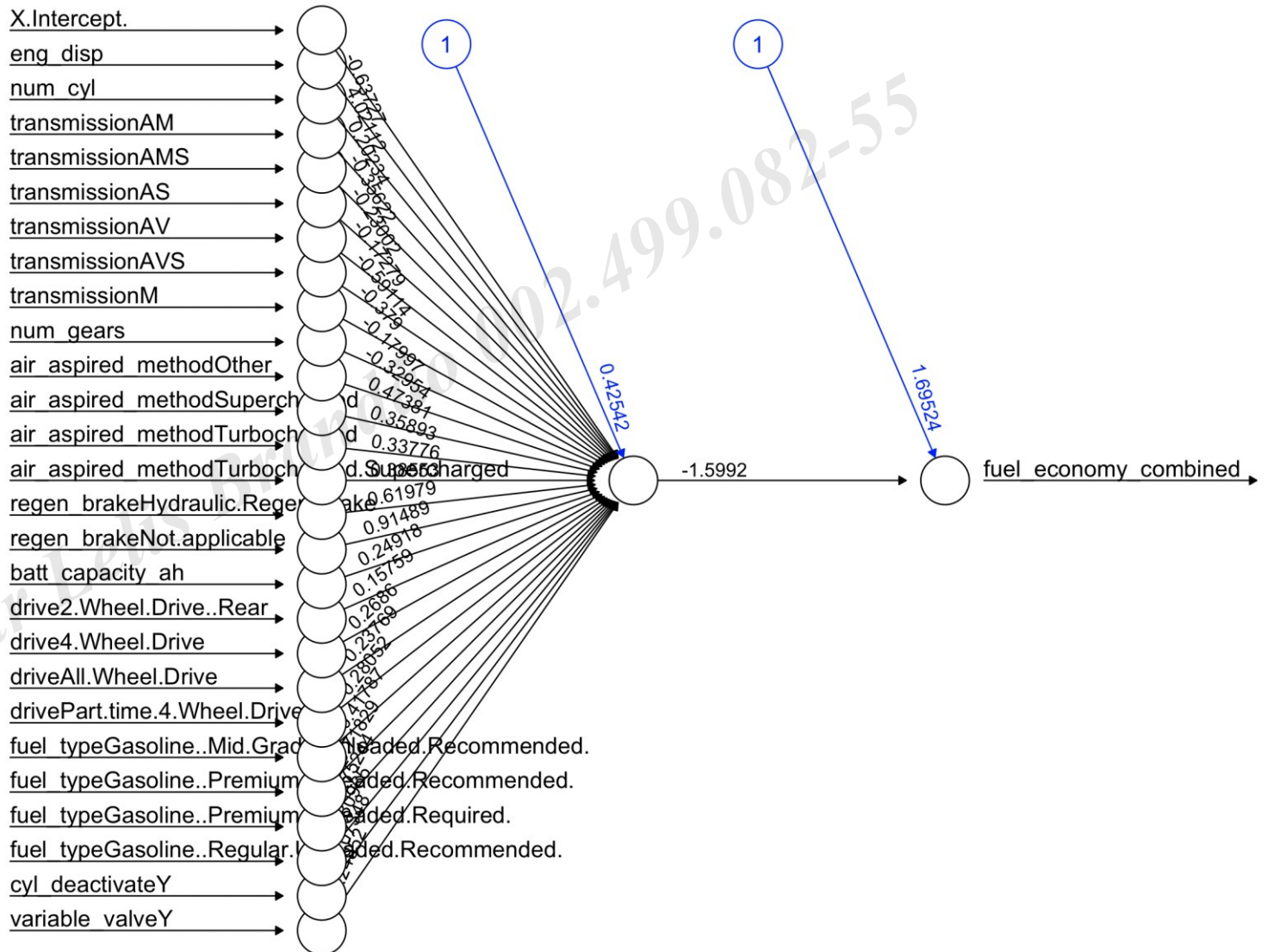
Previsão de consumo de veículo

- Tamanho do motor
- Combustível
- Número de cilindros
- Marca
- Potência
- Tração



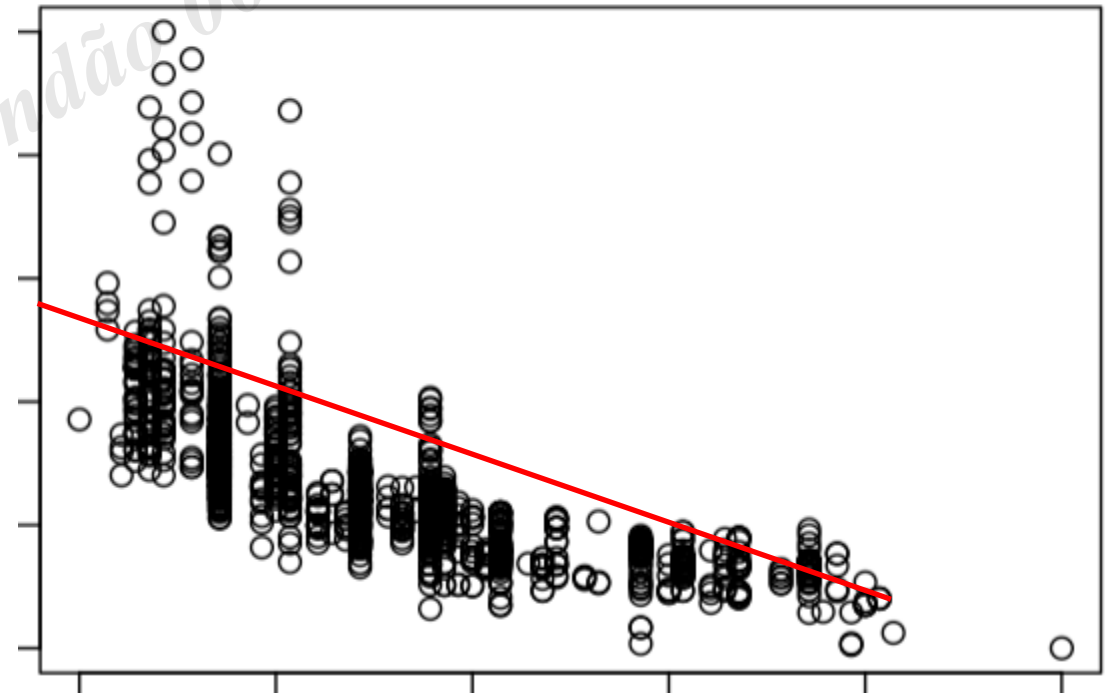
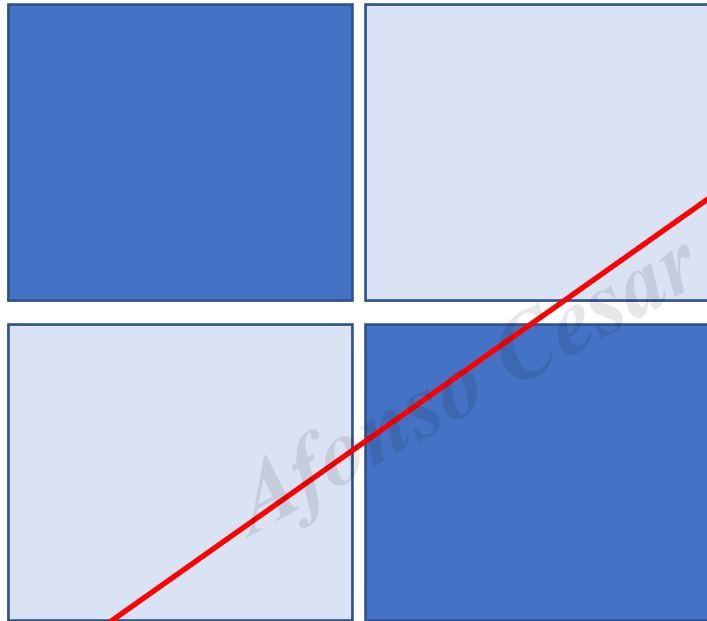
Perceptron Linear

- Possui a mesma estrutura de uma regressão linear com a função de ativação indicada.



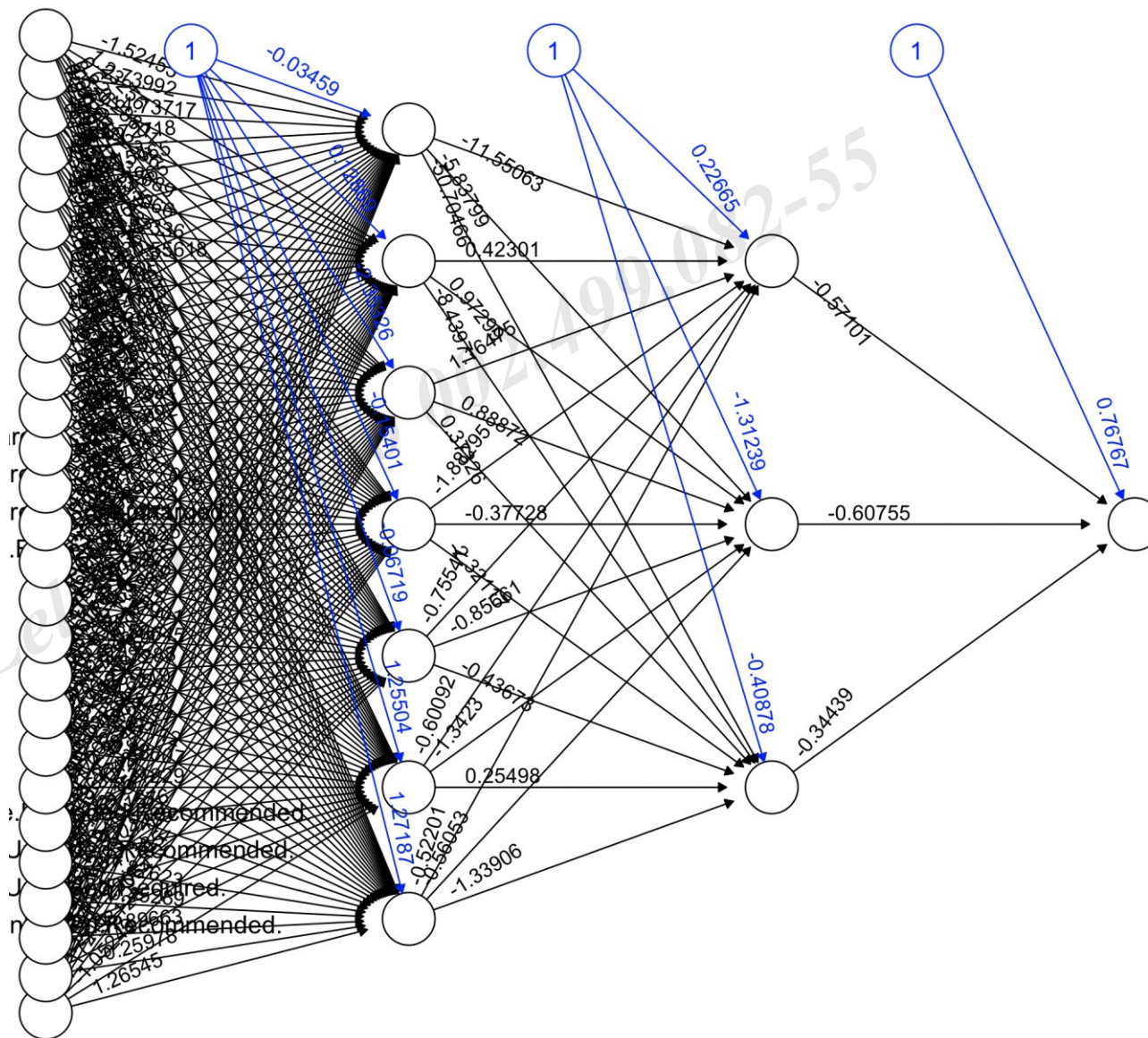
Limitações do perceptron linear

- O perceptron linear só captura padrões lineares



Perceptron multi-camada

- Possui camadas “escondidas” intermediárias
- Captura padrões não lineares
- Pode se aproveitar do processamento paralelo de GPUs
- Não é “interpretável” como a regressão





AREM4_script02_neural_networks.R

Funções de perda

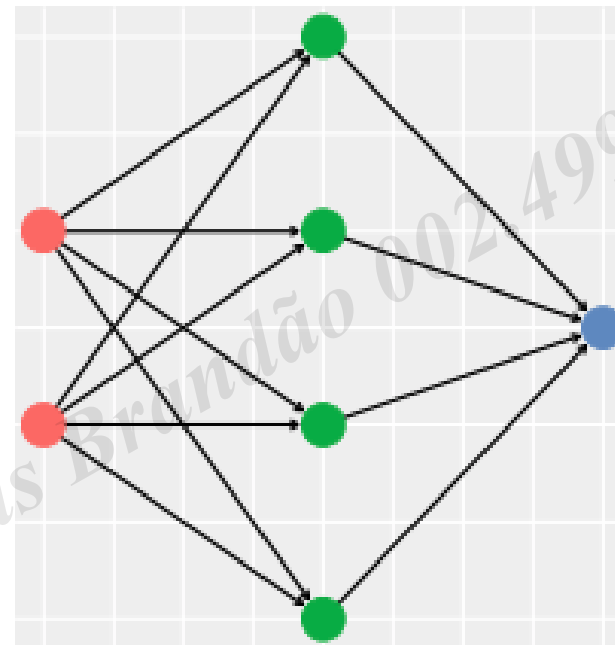
Variáveis Contínuas
SQE

$$SQE = \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

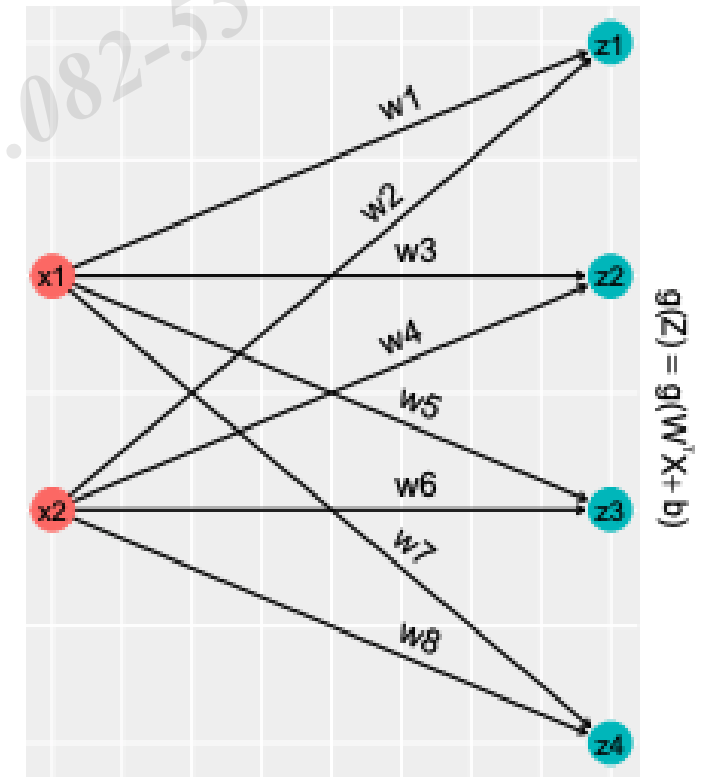
Variáveis binárias
Cross-Entropy

$$L = y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)$$

Redes Neurais Artificiais



layer ● Input ● Hidden ● Output



$$(q + X'M)\delta = (Z)\delta$$

Deep learning with R - Abhijit Ghatak, ed. Springer, 2019

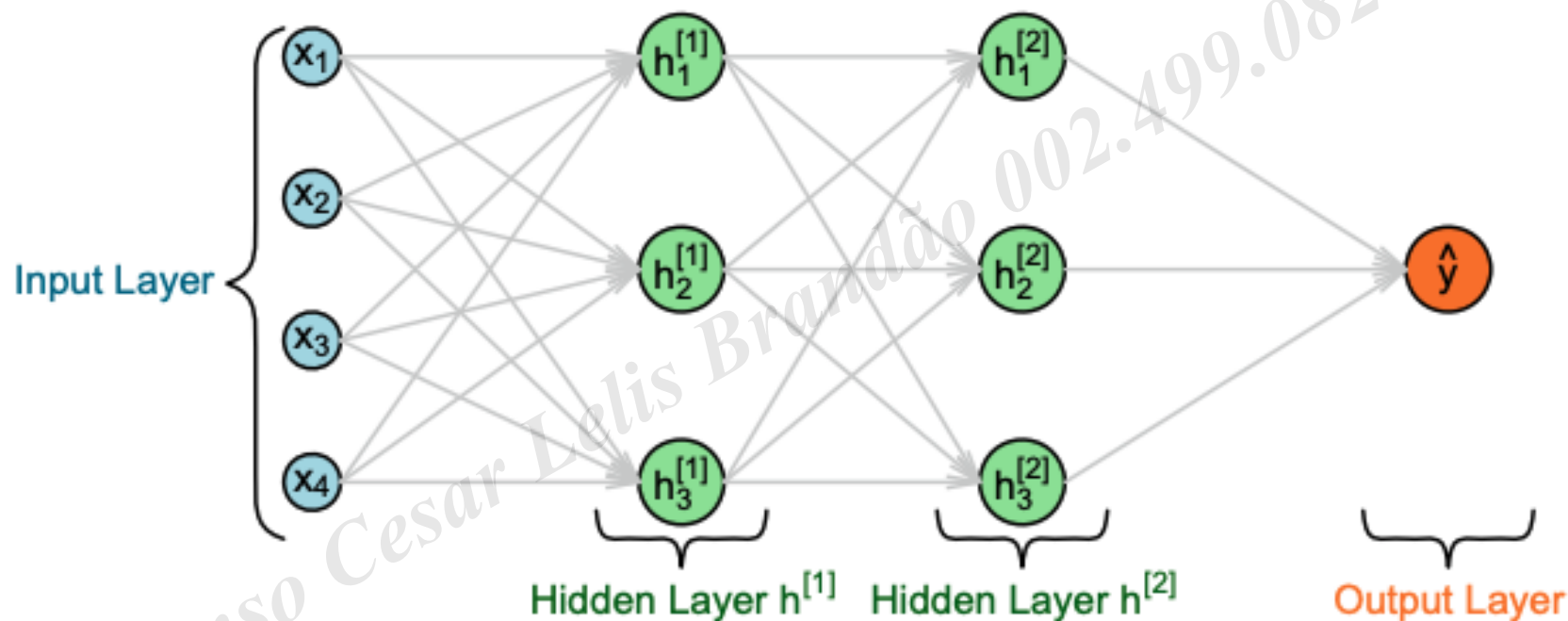
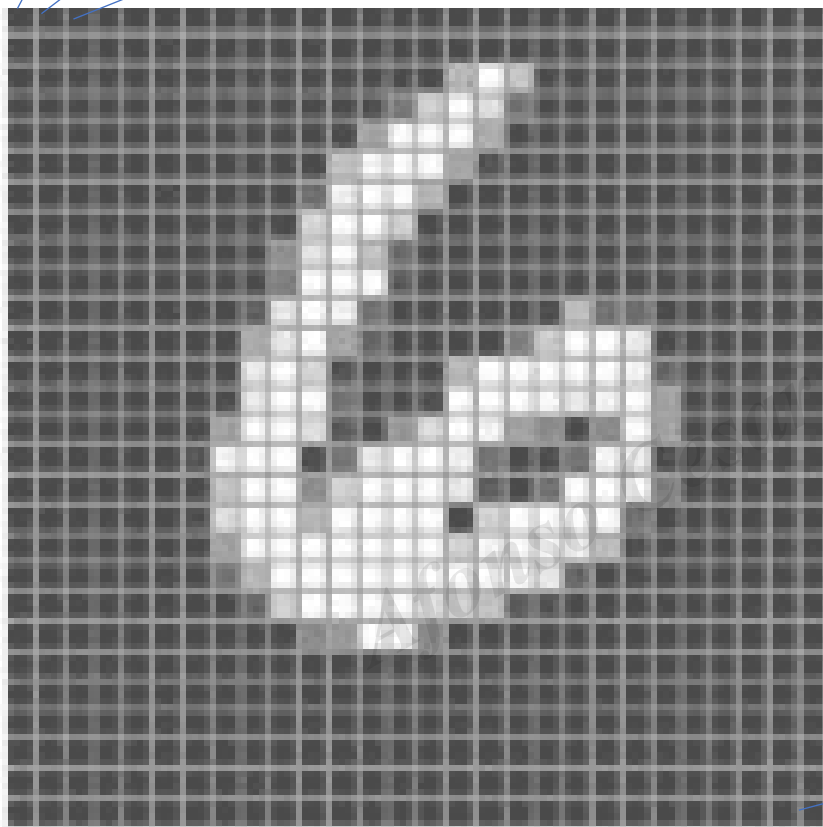


Fig. 2.3 A representation of a neural network with four input features, two hidden layers with three nodes each, and an output layer

Deep learning with R - Abhijit Ghatak, ed. Springer, 2019

Exemplo em OCR (MNIST)

Pixel 1
Pixel 2
Pixel 3



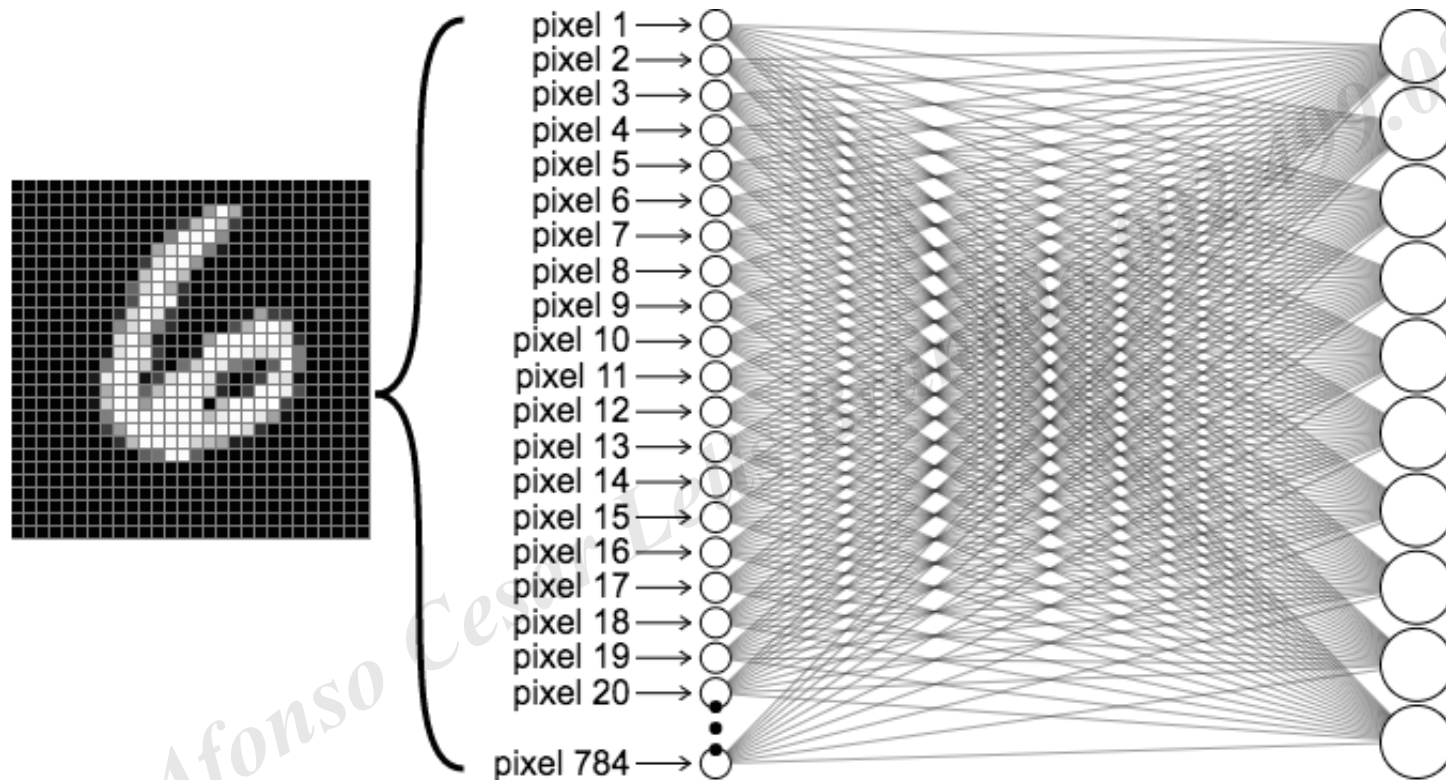
60.000 dessas

Viram uma
tabela assim

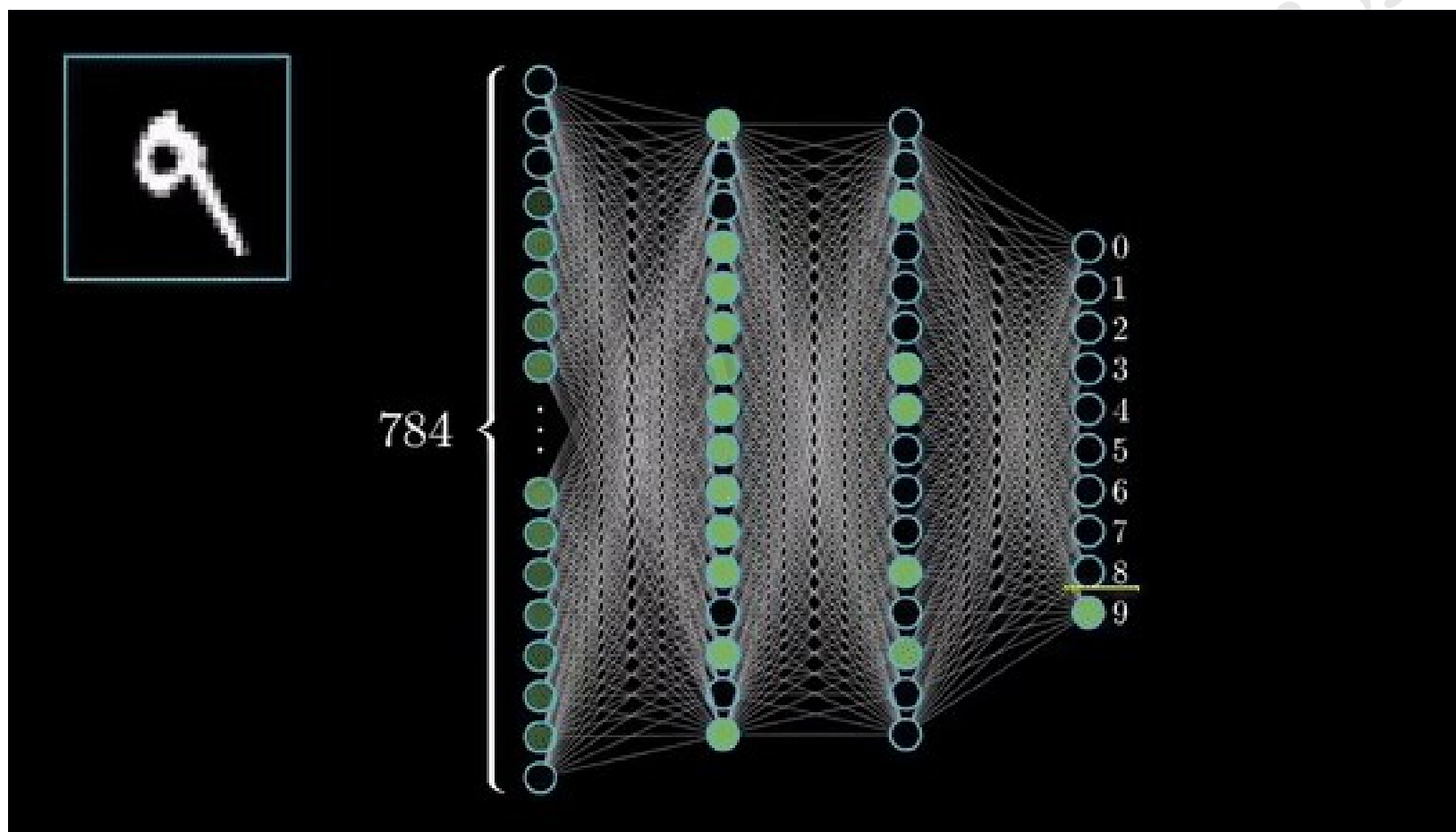
	Pixel 1	Pixel 2	Pixel 3	⋮	Pixel 784	Rótulo
Imagem 1	0	0	0...		0	4
Imagem 2	0	0	0...		0	3
Imagem 3	0	0	0...		0	9
⋮	⋮	⋮	⋮	⋮	⋮	⋮
Imagem 60.000	0	0	0...		0	5

Pixel 784

Rede Neural MNIST



Com apenas uma camada já temos $784 \times 10 = 7.840$ parâmetros!



3blue1brown - <https://www.youtube.com/watch?v=aircAruvnKk>

Gradiente descendente

É o algoritmo mais popular para se treinar redes neurais artificiais por apresentar algumas características:

- Pode alterar as estimativas com pequenos subconjuntos de pontos a cada iteração (no limite 1 único ponto)
- Não depende de inverter matriz
- Funciona com uma base de dados bem grande
- Pode ser processado em paralelo com GPU
- Permite interromper o algoritmo a certo ponto e continuar mais tarde ou em outro problema semelhante (*transfer learning*)

Redes Gradiente Descendente

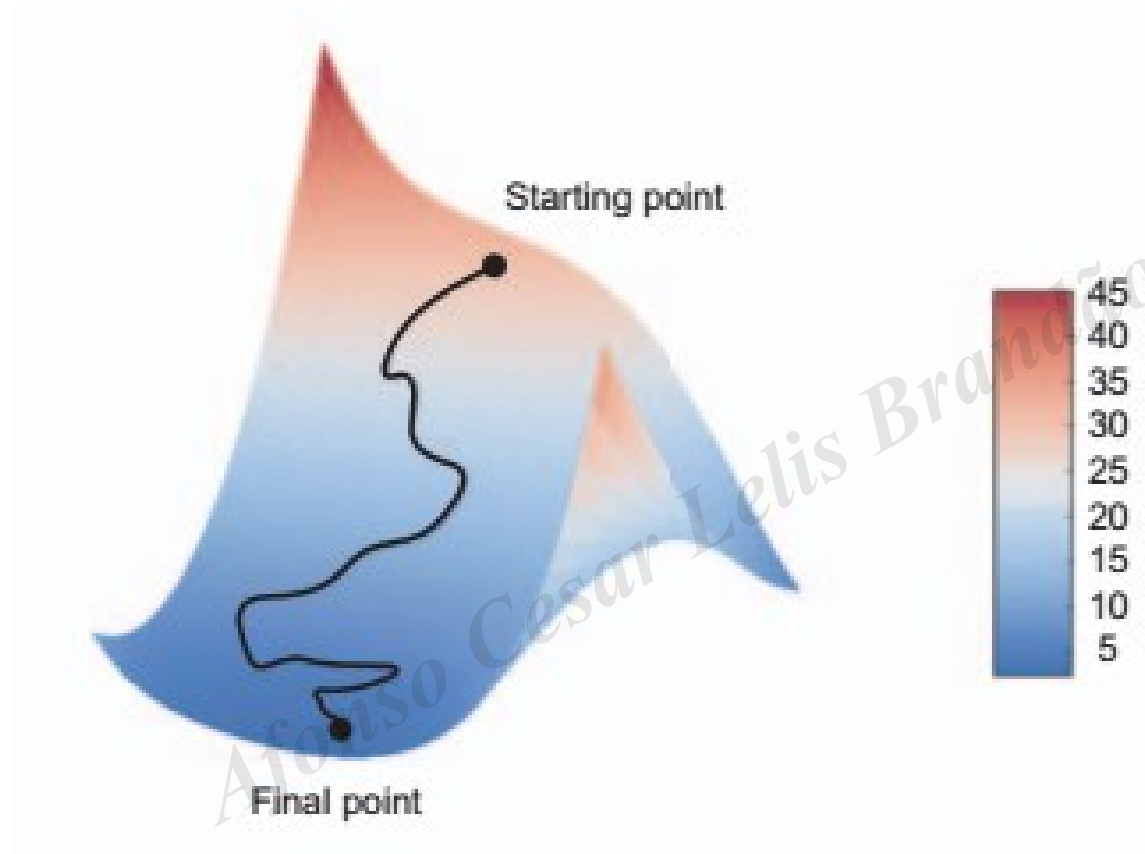


Figure 2.12 Gradient descent down a 2D loss surface (two learnable parameters)

Deep learning with python – François Chollet

Gradiente descendente

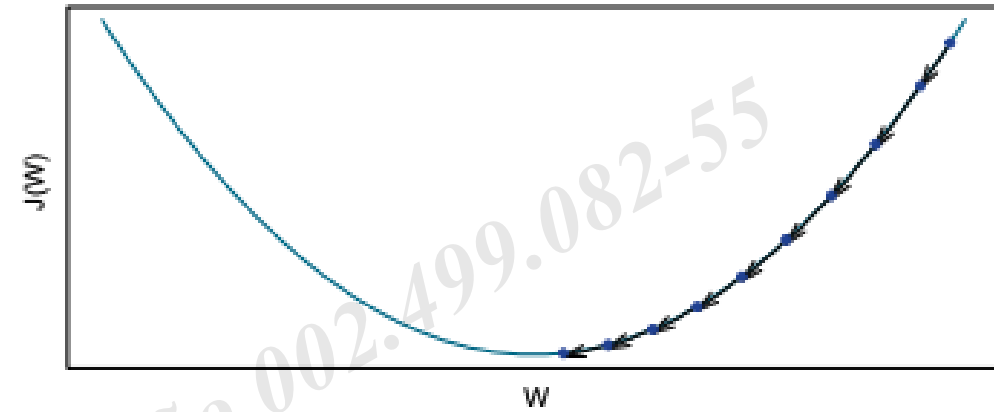


Fig. 1.4 Gradient descent: Rolling down to the minima by updating the weights by the gradient of the loss function

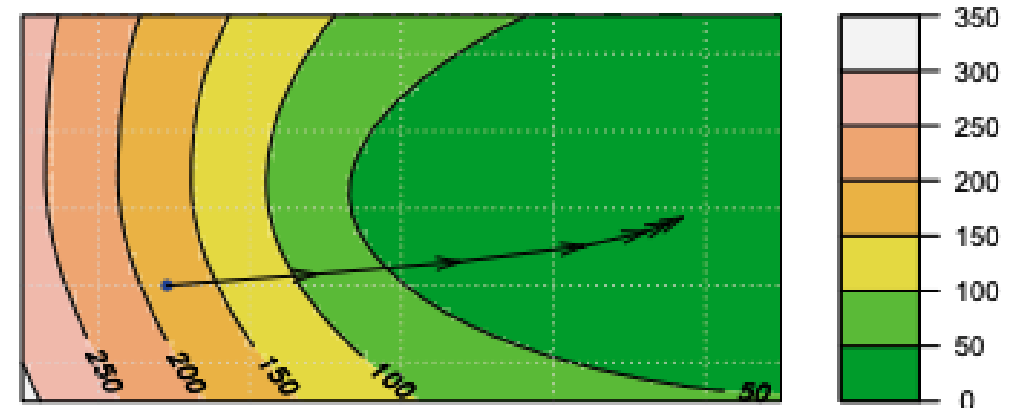


Fig. 1.5 A contour plot showing the cost contours of a sigmoid activation neural network and the cost minimization steps using the gradient descent optimization function

Demora pra rodar?

Uma breve história

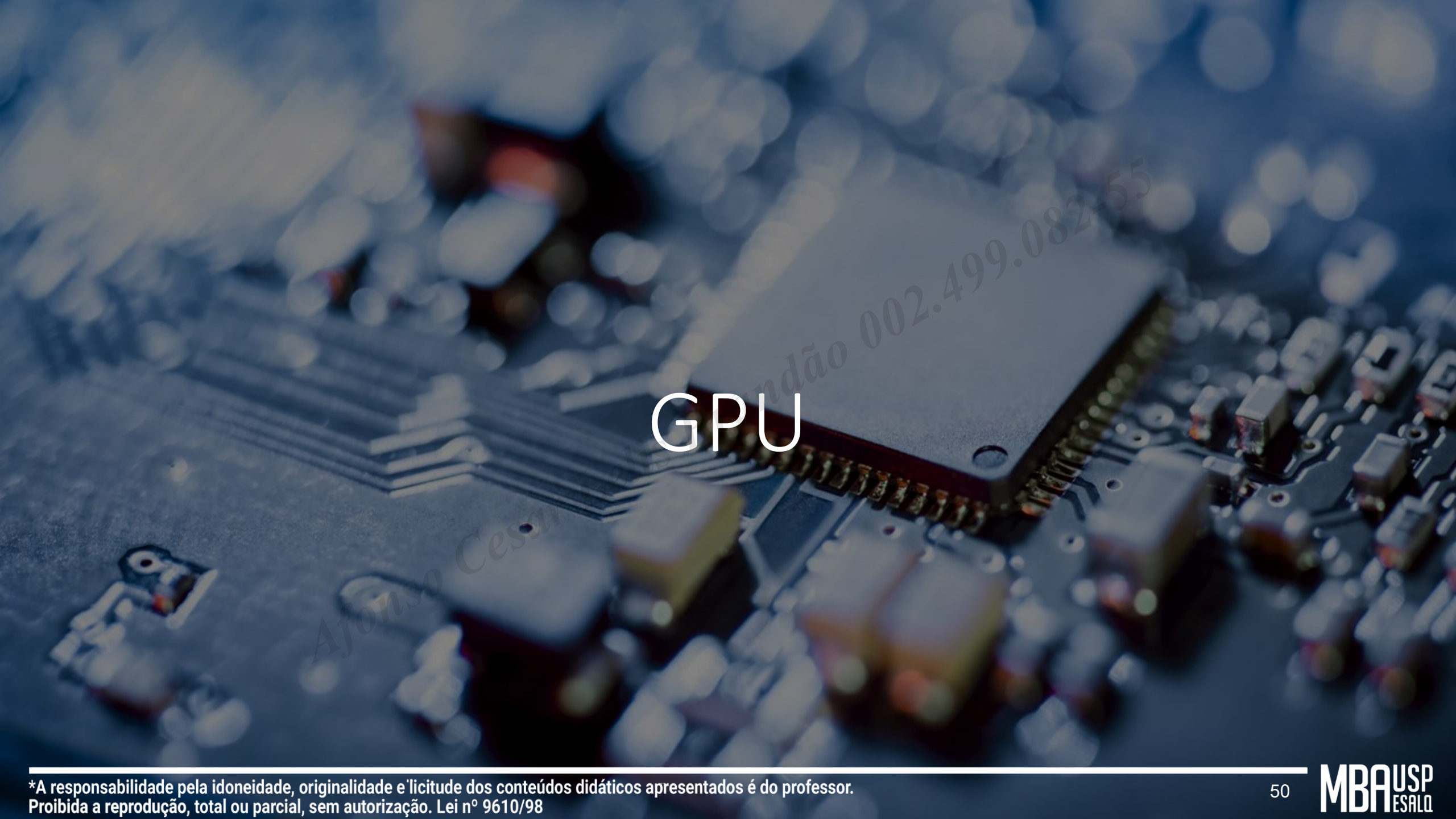


game

A contribuição da indústria do Game nas Redes Neurais

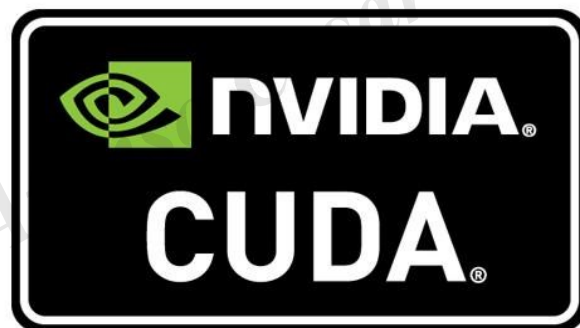
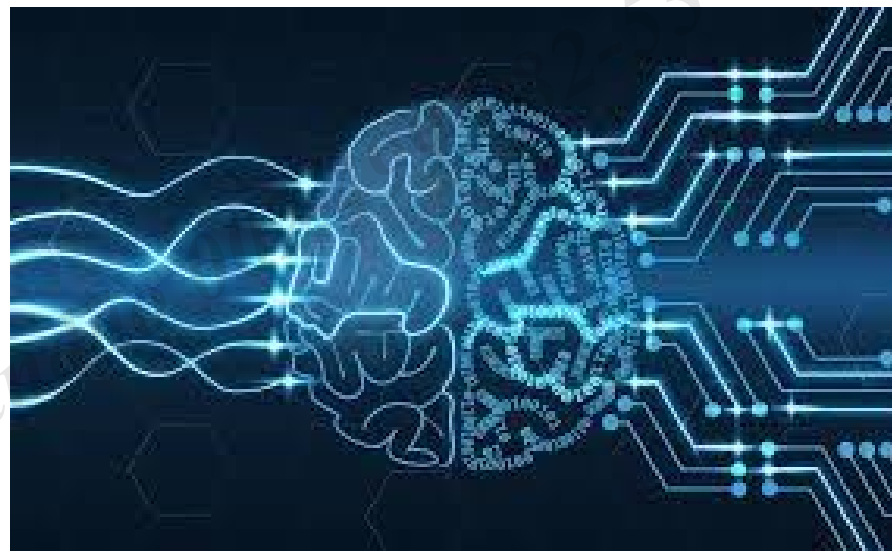
Processadores

- Distância entre transístores: 14 nm
- Fio de cabelo humano: 80.000 nm
- Diâmetro do átomo de ouro: 0,3 nm

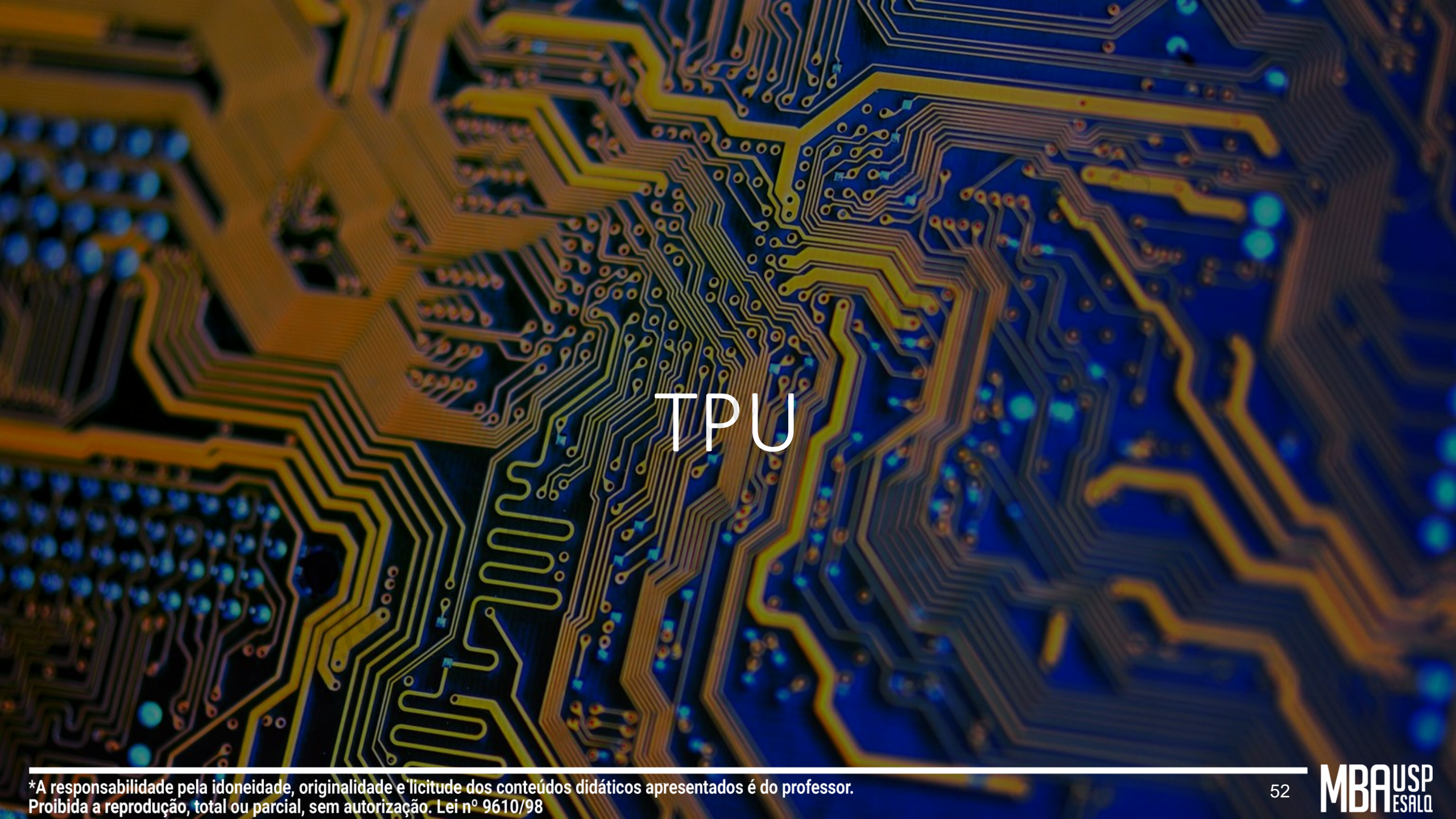
A close-up, shallow depth-of-field photograph of a GPU chip mounted on a circuit board. The chip is a dark, square component with gold-plated pins along its edges. The surrounding circuit board is populated with various other components, including capacitors and smaller chips, all slightly out of focus. The overall color palette is cool, dominated by blues and greys.

GPU

Processamento com GPU



AMD
ROCm



TPU

Regularização L2

Variáveis Contínuas
SQE

$$SQE = \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \lambda \sum \beta_i^2$$

Variáveis binárias
Cross-Entropy

$$L = \sum y_i \log(\hat{y}_i) + \lambda \sum \beta_i^2$$



Reconhecimento de atividade humana com smartphone



AREM4_script03_rna_har.R

Conclusões

- Redes Neurais são a introdução ao Deep Learning (que é um ramo muito promissor)
- São poderosas e flexíveis
- Requerem poder computacional especial (GPU / TPU)
- São famosas em dados menos estruturados (ex: imagens, áudio)





“Me enganei apenas uma vez, quando pensei estar enganado”

Prof. Girafales



Por hoje é só ;)



[linkedin.com/in/joao-serrajordia](https://www.linkedin.com/in/joao-serrajordia)

OBRIGADO

[linkedin.com/in/joao-serrajordia](https://www.linkedin.com/in/joao-serrajordia)